

BuildFlow

Modular Monolith Architecture

BuildFlow will be a construction operations platform. This document defines the architecture before implementation begins. It fixes the module boundaries, states what each module will own, and defines the interfaces through which modules will reach each other. It also draws the MVP line: nine modules will be built for the private pilot, and no MVP module depends on one that is not yet built.

9

modules in the MVP
6 business, 3 platform

7

modules deferred
Release 2 and later

4

roles in the MVP
owner, PM, supervisor,
accountant

RLS

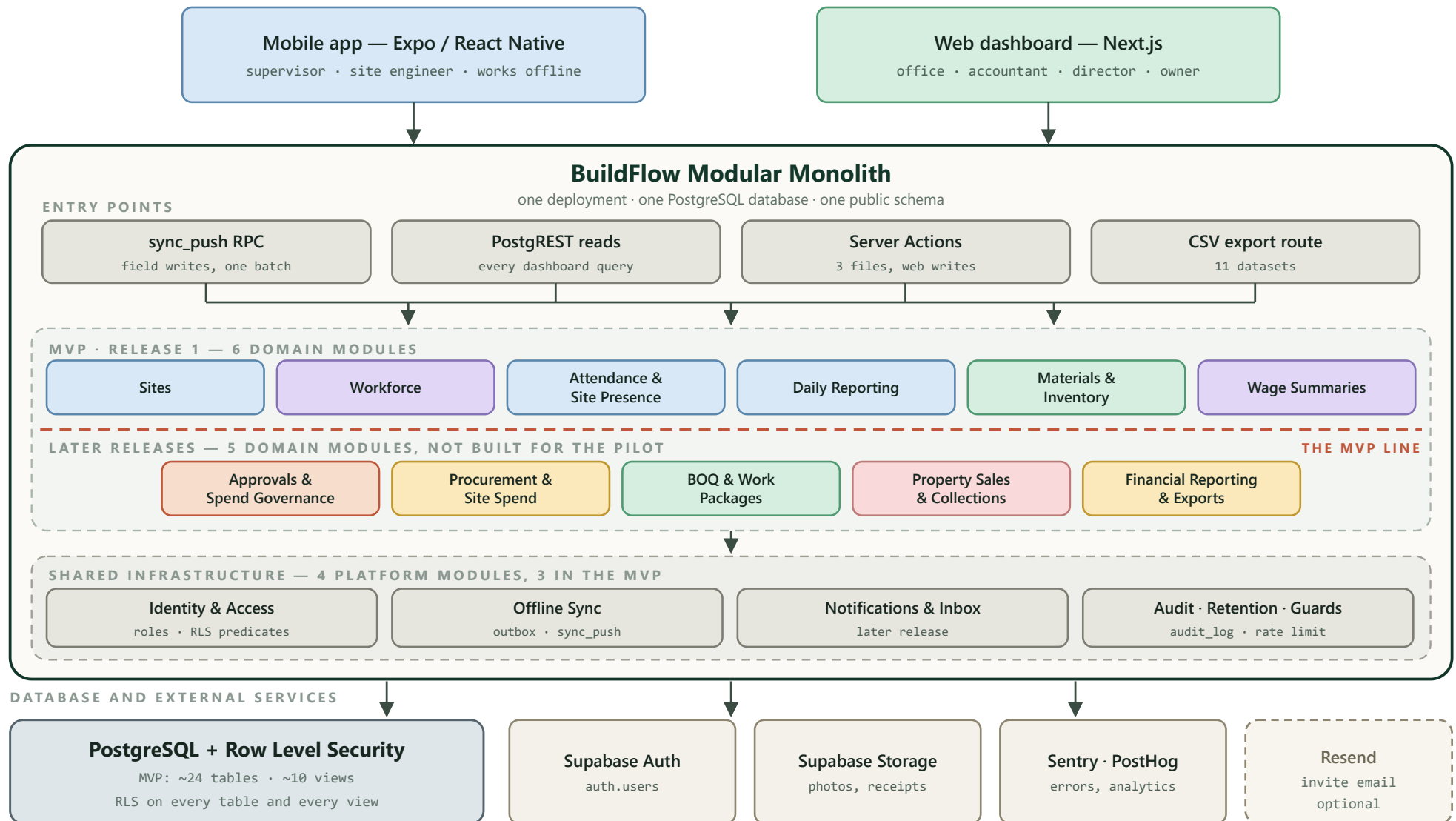
authorization boundary
enforced in the database

Contents

Architectural approach and system architecture	02
Module boundaries	03
Module dependencies	04
Domain and database ownership	05
Cross-module communication	06
Module interfaces	07
Identity, authorization and shared infrastructure	08
Offline and mobile architecture	09
Key workflows, rules and build sequence	10

Architectural Approach and System Architecture

The system will be built as one deployment over one PostgreSQL database. The code will be divided into modules, of which nine will be built for the MVP. Two clients will use the same backend, and every request will run as the signed-in user.



Module Boundaries

Nine modules will be built for the MVP. Each will own its tables and be entered only by name. Seven more are defined so the boundaries hold later.

The nine MVP modules

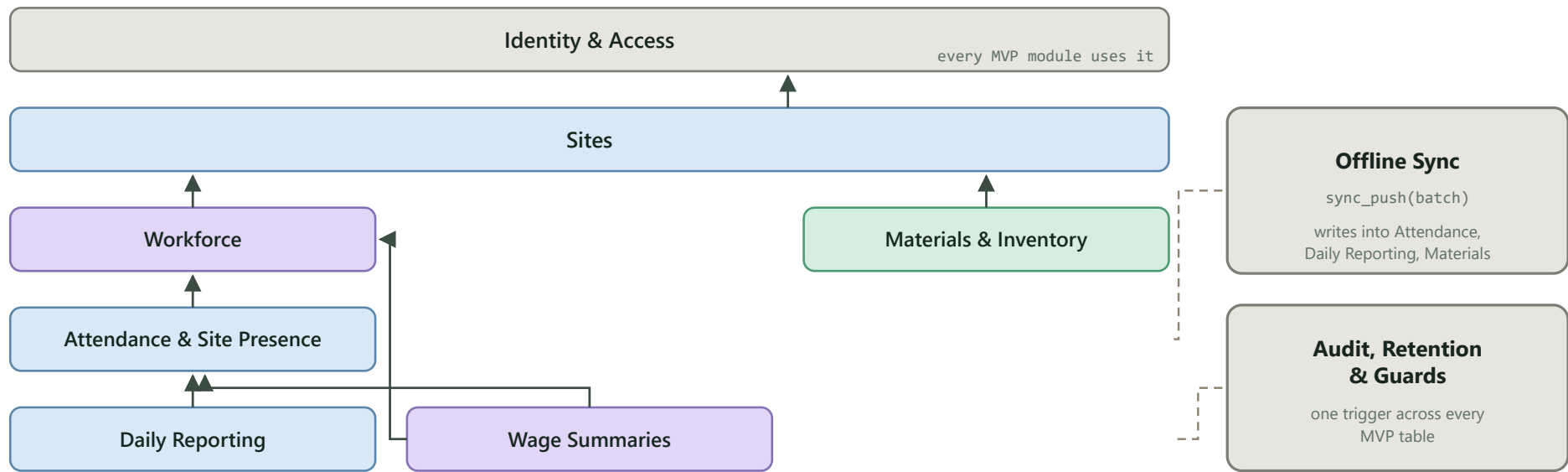
Module	Responsibility	Tables it will own
Identity & Access platform	Will hold organisations, members, roles and invites.	organizations, profiles, memberships, invites
Sites	Will hold sites and assign supervisors and managers to them.	sites, site_assignments
Workforce	Will hold workers, their wage rates and their consent records.	workers, worker_site_assignments, worker_wage_rates
Attendance & Site Presence	Will record check-in, the daily roster and corrections.	site_visits, attendance_days, attendance_entries, attendance_corrections
Daily Reporting	Will record the daily site report, its blockers and its safety flag.	daily_reports, daily_report_revisions, report_photos
Materials & Inventory	Will keep the stock ledger and compute balances and alerts.	materials, material_transactions
Wage Summaries	Will turn approved attendance into a wage summary.	wage_periods, wage_lines
Offline Sync platform	Will move field writes to the server exactly once.	device SQLite: outbox and 8 mirror tables
Audit, Retention & Guards platform	Will record privileged changes and enforce retention and rate limits.	audit_log, purge_log, rate_limit_counters

Deferred — defined, not built for the pilot

Module	What it will add	Release	Will depend on
Projects & BOQ	Projects above sites, a priced estimate by version, and actual use.	Release 3	Sites, Materials
Approvals & Spend Governance	The shared request-and-decide path and the spend threshold.	Release 3	Identity & Access
Procurement & Site Spend	Vendors, purchase orders, deliveries, expenses and payments.	Release 3	Approvals, Materials
Property Sales & Collections	Plots, bookings, receipts, brokers and broker payouts.	Release 3	Approvals
Financial Reporting & Exports	Project budget and funding, project spend, and P&L.	Release 3	Procurement, Wages, Sales
Punch List	Defects tracked beyond the blocker field on the daily report.	Release 4	Daily Reporting
Notifications & Inbox platform	One list of what is waiting for each user.	Release 2	Attendance, Reporting

Module Dependencies

An arrow means one module will need another. Each arrow will be a named call, a join, a trigger or a foreign key — nothing else. Every arrow below stays inside the MVP set, so nothing waits on a module that will not be built.



Arrows point from a module to the module it needs. No arrow leaves the MVP set, so no module waits on one that will not be built.

Dependencies point downward.

Identity, then Sites, then field capture, then wages.

No MVP module looks ahead. Nothing

in the MVP calls a module from a later release.

Review stays inside its module.

Attendance and Daily Reporting each approve their own records.

One shared approval engine waits. It

arrives in Release 3, with the first money that needs it.

Domain and Database Ownership

Each business area will have exactly one owning module. No table will be owned by two modules.

Business area	MVP module	Tables it will own	Views it will publish
Organisation and people	Identity & Access	organizations, memberships, invites	—
Work containers	Sites	sites, site_assignments	site_timeline
Labour supply	Workforce	workers, worker_wage_rates	worker_retention_status
Site presence and labour record	Attendance & Site Presence	site_visits, attendance_days, attendance_entries	attendance_day_summary, site_exceptions
Site record	Daily Reporting	daily_reports, report_photos	daily_report_status
Stock	Materials & Inventory	materials, material_transactions	material_balances, material_alerts
Labour cost	Wage Summaries	wage_periods, wage_lines	wage_summary
Field reliability	Offline Sync	device outbox	—
Accountability and retention	Audit, Retention & Guards	audit_log, purge_log, rate_limit_counters	—

All tables will live in one `public` schema. Ownership will be a rule the team follows, held in place by narrow entry points and review rather than by separate schemas. Every view will be `security_invoker`, so a view will never expose more than its underlying tables allow.

Cross-Module Communication

Modules will communicate in five ways only. Every path will be synchronous: no message queue, no event bus, no second service.

Permission check

Every table's RLS policy will call a named function.

```
is_org_member() ·  
has_site_access()
```

Named function

One module will call a function the owning module publishes.

```
record_site_expense() ·  
confirm_po_delivery()
```

Database trigger

A write in one module will cause a write in another.

```
record_boq_actual on  
material_transactions
```

Published view

One module will read another only through a view.

```
project_spend · vendor_ledger  
· notifications
```

Foreign key

A row will point at a row owned by another module.

```
issues.daily_report_id
```

Inside the MVP — every path that will exist at pilot

From	To	Mechanism	Purpose
Every MVP module	Identity & Access	Permission check inside an RLS policy	Decide which rows a user may see or change
Every MVP module	Audit, Retention & Guards	One generic trigger across the MVP tables	Record every privileged change
Offline Sync	Attendance, Reporting, Materials	Direct insert from <code>sync_push</code>	Field capture; these tables will be append-only
Attendance, Workforce	Sites	Foreign key to <code>sites</code> , checked by <code>has_site_access</code>	A record will always belong to one site
Daily Reporting	Attendance	Device-local read of the day's roster	Headcount will be pre-filled rather than retyped
Wage Summaries	Attendance, Workforce	Direct read of approved days and rates	Wages will come from approved days only

Deferred paths, listed so the MVP does not pre-build them

From	To	Mechanism	Arrives in
Procurement, Property Sales, Projects & BOQ	Approvals	Will call <code>request_approval()</code>	Release 3
Approvals	Procurement, Sales, Projects & BOQ	Direct update behind a session flag	Release 3
Offline Sync	Procurement & Site Spend	Named function, never a direct insert	Release 3
Procurement	Materials & Inventory	One stock row per delivered line	Release 3
Materials	Projects & BOQ	Trigger on material use	Release 3
Financial Reporting	Procurement, Wage Summaries, Sales	Reads of published views	Release 3

Module Interfaces

Each module will expose a fixed surface: the functions other modules may call, and the views other modules may read. Nothing outside this list will be reachable across a boundary.

Module	Functions it will expose	Views it will publish
Identity & Access	is_org_member · is_org_owner · has_site_access · can_record_on_site · can_review_site · can_view_wages · create_invite · accept_invite	—
Sites	archive_site	site_timeline · sites.id
Workforce	import_workers · anonymize_worker	worker_retention_status · effective wage rate
Attendance & Site Presence	review_attendance_day · review_attendance_correction	approved attendance_days · site_exceptions
Daily Reporting	review_daily_report	daily_report_status
Materials & Inventory	apply_material_transaction	material_balances · material_alerts
Wage Summaries	calculate_wage_period · lock_wage_period · reopen_wage_period	wage_summary
Offline Sync	sync_push(batch)	—
Audit, Retention & Guards	write_audit · enforce_rate_limit · purge_expired_non_statutory	—

Nothing outside these lists will be reachable across a boundary. The same rule will hold for the deferred modules when they arrive: the spend tables of Release 3 will carry no insert policy at all, so a named function will be the only way in.

Identity, Authorization and Shared Infrastructure

Row Level Security will be the authorization boundary. It will run inside PostgreSQL, so every client will get the same answer.



Privileged writes do not bypass this.
They will go through narrow SECURITY DEFINER functions that check the caller themselves. The service-role key will be used nowhere in application code.

Four roles in the MVP

Role	What it may do
owner	Everything, on every site. Only an owner may grant the owner role, and only an owner may reopen a locked wage period.
project_manager	Reviews and approves site work. Manages sites and workers. May not read wage amounts.
supervisor	Records work on assigned sites. Sees no wage data.
accountant	Builds and exports wage summaries. May lock a wage period, not reopen one.

Each role will be verified by running every operation as every role against the database before the module is accepted.

Shared infrastructure

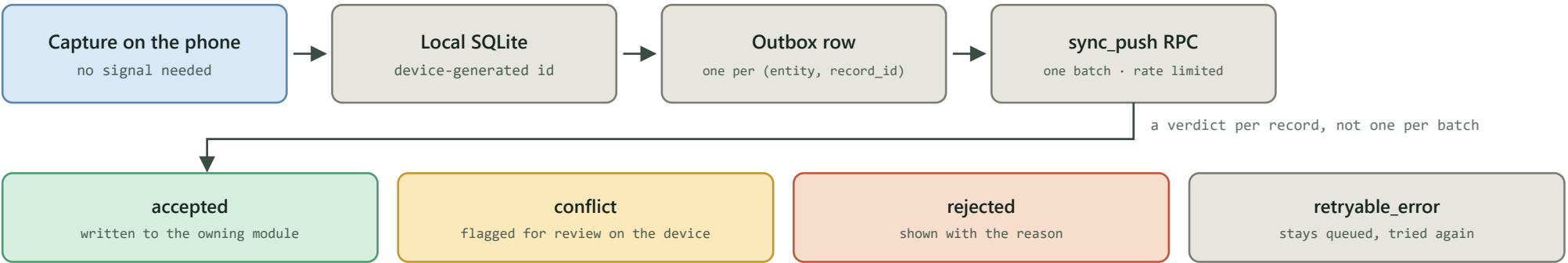
Platform module	What it will provide
Identity & Access	Four roles, membership, and the named permission checks every RLS policy will call.
Offline Sync	One RPC carrying seven entity types from the phone, returning a verdict per record.
Audit, Retention & Guards	One audit trigger across every MVP table, a retention purge, and per-organisation rate limits.

Roles deferred

Role	Arrives in	Why it waits
admin	Release 2	Organisation administration delegated from the owner.
director	Release 3	Read-only visibility across every site.
site_engineer	Release 3	A review tier between supervisor and manager.

Offline and Mobile Architecture

The mobile app will work with no signal. Every write will be stored on the device first and sent later.



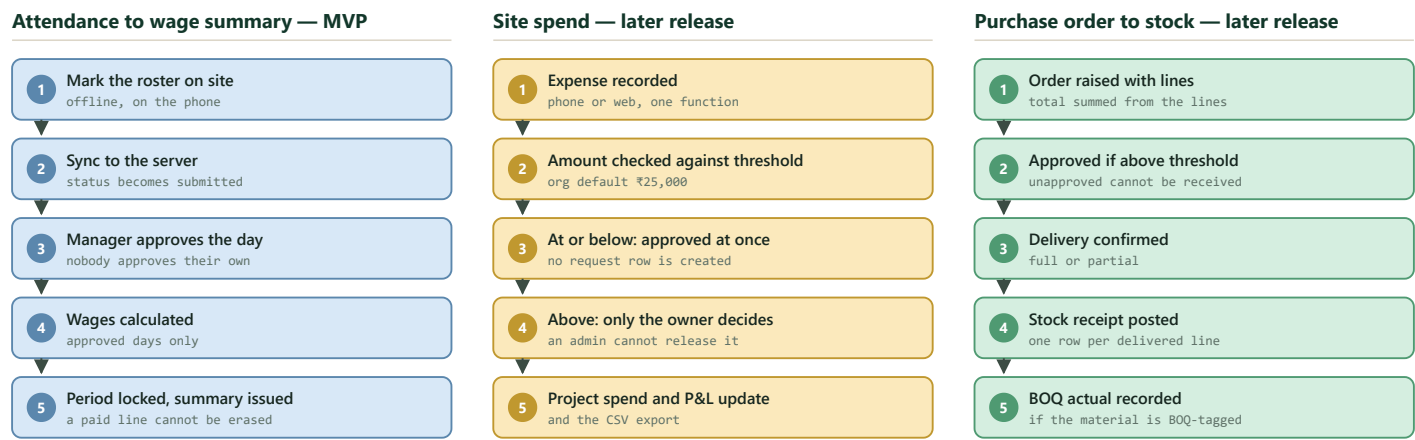
A repeated push does not duplicate. One bad record does not block the rest of its batch. The app never shows “submitted” before the server confirms. Photos and receipts upload separately to Supabase Storage. An image never blocks a record. Spend and purchase orders call named functions instead of inserting.

How the record will be written	Entities carried by sync_push	Why
Direct insert MVP	site_visits · attendance_days · attendance_entries · attendance_corrections · daily_reports · report_photos · material_transactions	These eight tables will be append-only, or safe to repeat on the same device id.
Named function Release 3	site_expenses · purchase_orders · po_deliveries	Money must pass the spend threshold, so the phone will call the same function the web does. No money entity reaches the phone in the MVP.

Pull will need no custom code: PostgREST already serves the cursor protocol under Row Level Security. Only push will need code, and it will be a PostgreSQL function rather than a second service to deploy.

Key Workflows, Architecture Rules and Build Sequence

Three paths will cross module boundaries end to end — one in the MVP, two later. The rules keep those boundaries intact, and the build sequence orders the work so no module begins before the ones it depends on.



Rules for keeping the boundary

- 1. One writer per table. To change another module's data, call that module's function.
- 2. No MVP module may call, join to or read a module from a later release.
- 3. Keep ledgers append-only. Every guard must still allow customer offboarding.
- 4. Depend downward. A new arrow pointing up is a decision, not a shortcut.
- 5. Read another module only through a published view, and keep the view `security_invoker`.
- 6. Keep `sync_push` a thin router. Logic placed there will run only on phones.
- 7. Name every permission check. No policy should compare a role inline.
- 8. Add a role only when a user cannot do their job without it. The MVP has four.

Build sequence

STEP 1 MVP	Foundation Identity & Access · Audit, Retention & Guards · Sites
STEP 2 MVP	Field capture Workforce · Attendance · Daily Reporting · Materials · Offline Sync
STEP 3 MVP	Close the loop Wage Summaries · dashboard and approvals · exports
THE MVP LINE	Private pilot runs here, then the gate Release 2, then Release 3 – nothing below starts first